

S26CS6.401 - Software Engineering

Take Home Activity 2 — Design Patterns Implementation

Team No: 28

Team Members:

- Tanishq Agarwal (2022101060)
- Daksh Shah (2022101068)

- Gaurav Goswami (2025204014)
- Shefali Bajaj(2025801015)

Repository: [Reservation-System-Starter](#)

References:

1. [Professor's Design Patterns Examples](#)
 2. [Refactoring Guru — Design Patterns](#)
-

Assumptions

1. The codebase under analysis is the unmodified `master` branch of the [Reservation-System-Starter](#) repository.
 2. All design patterns are applied independently — each pattern section shows the before/after for that specific refactoring.
 3. Existing test behavior is preserved; refactoring does not change external functionality.
 4. Java is the implementation language, consistent with the original codebase.
 5. Class diagrams use UML notation rendered via PlantUML.
-

1. Strategy Pattern — Payment Processing

1.1 Problem Identification

In the current codebase, `FlightOrder.java` contains **hardcoded payment logic** for two payment methods — Credit Card and PayPal. The relevant methods are:

- `processOrderWithCreditCard(CreditCard creditCard)`
- `processOrderWithCreditCardDetail(String number, Date expirationDate, String cvv)`
- `processOrderWithPayPal(String email, String password)`
- `payWithCreditCard(CreditCard card, double amount)`
- `payWithPayPal(String email, String password, double amount)`

Each payment method has its own dedicated process and pay methods directly inside `FlightOrder`. This means:

- **Adding a new payment method** (e.g., Bitcoin, UPI, Bank Transfer) requires **modifying** `FlightOrder` — violating the **Open/Closed Principle**.
- Payment logic is **tightly coupled** to the order class, making it hard to test payment methods independently.
- The `FlightOrder` class takes on **too many responsibilities** — both order management and payment processing — violating the **Single Responsibility Principle**.

Reference: [Strategy Pattern — Refactoring Guru](#)

1.2 Pattern Description

The **Strategy Pattern** is a behavioral design pattern that defines a family of algorithms, encapsulates each one in a separate class, and makes them interchangeable. The client (context) delegates the algorithm execution to a strategy object rather than implementing it directly.

Key participants:

- **Strategy (interface):** Declares the common interface for all payment algorithms.
- **Concrete Strategies:** Each implements the payment interface with a specific algorithm (CreditCard, PayPal, etc.).
- **Context:** The `FlightOrder` class holds a reference to a Strategy and delegates payment to it.

Reference: [Professor's Strategy Pattern Example — SortingStrategy](#)

1.3 Application and Reasoning

Why Strategy Pattern fits here:

The payment processing in `FlightOrder` is a classic case for the Strategy Pattern because:

1. There are **multiple algorithms** (CreditCard, PayPal) that achieve the same goal (processing payment).
2. The algorithms are **selected at runtime** — the customer chooses how to pay.
3. New payment methods will likely be added in the future.

How it is applied:

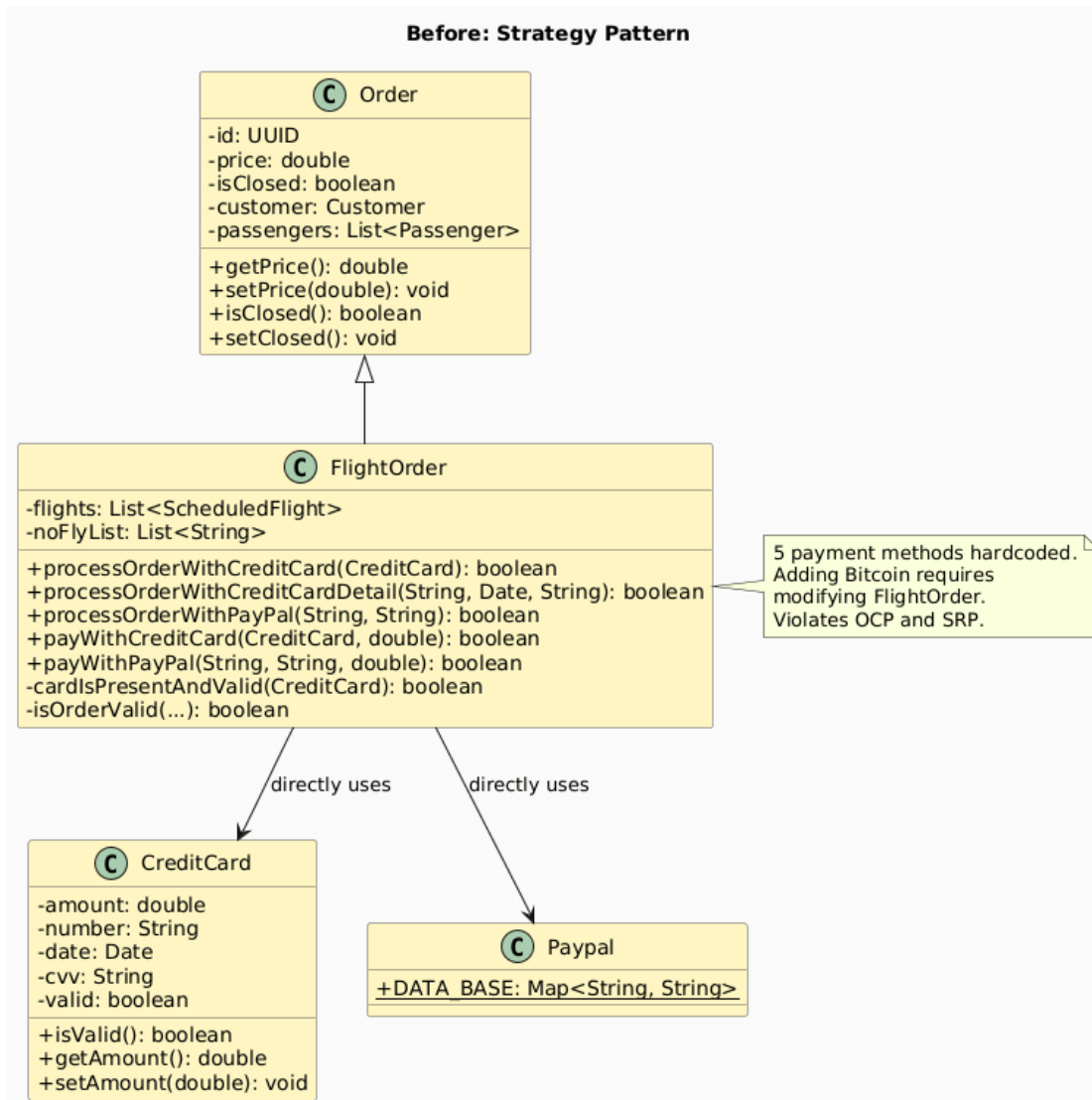
- Extract a `PaymentStrategy` interface with a single method `pay(double amount)`.
- Create `CreditCardPayment` and `PayPalPayment` as concrete strategy classes.
- `FlightOrder` holds a `PaymentStrategy` reference and delegates payment to it via `processOrder()`.

1.4 Benefits and Drawbacks

Benefits	Drawbacks
Open/Closed Principle: New payment methods can be added without modifying <code>FlightOrder</code> .	Increases the number of classes in the project.
Single Responsibility: <code>FlightOrder</code> focuses on order logic; payment logic is separated.	Clients must be aware of different strategies to select the right one.
Testability: Each payment strategy can be unit-tested independently.	Slight overhead for simple cases with only 2 strategies.

Runtime flexibility: Payment method can be swapped at runtime.

1.5 Class Diagram — Before



```
@startuml
title Before Strategy Pattern

class Order {
    - id: UUID
    - price: double
    - isClosed: boolean
    - customer: Customer
    - passengers: List<Passenger>
    + getPrice(): double
}
```

```

+ setPrice(double): void
+ isClosed(): boolean
+ setClosed(): void
}

class FlightOrder extends Order {
- flights: List<ScheduledFlight>
- noFlyList: List<String>
+ processOrderWithCreditCard(CreditCard): boolean
+ processOrderWithCreditCardDetail(String, Date, String): boolean
+ processOrderWithPayPal(String, String): boolean
+ payWithCreditCard(CreditCard, double): boolean
+ payWithPayPal(String, String, double): boolean
- cardIsPresentAndValid(CreditCard): boolean
- isOrderValid(...): boolean
}

class CreditCard {
- amount: double
- number: String
- date: Date
- cvv: String
- valid: boolean
+ isValid(): boolean
+ getAmount(): double
+ setAmount(double): void
}

class Paypal {
+ {static} DATA_BASE: Map<String, String>
}

FlightOrder --> CreditCard : directly uses
FlightOrder --> Paypal : directly uses

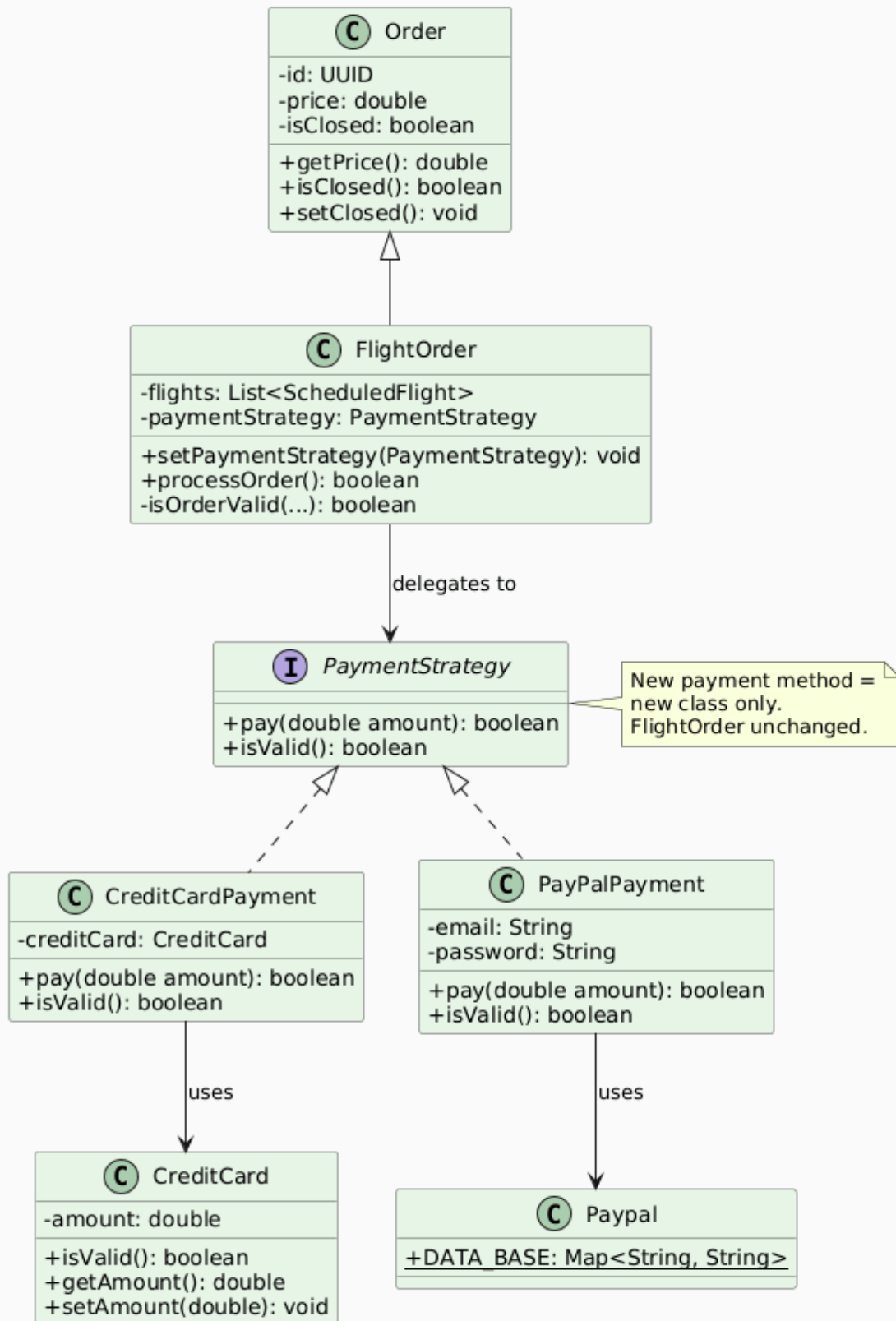
@enduml

```

Problem: FlightOrder is **directly coupled** to both CreditCard and Paypal classes. All payment logic lives inside FlightOrder .

1.6 Class Diagram — After

After: Strategy Pattern



```

@startuml
title After Strategy Pattern

class Order {
    - id: UUID
    - price: double
    - isClosed: boolean
    - customer: Customer
    - passengers: List<Passenger>
    + getPrice(): double
    + setPrice(double): void
    + isClosed(): boolean
    + setClosed(): void
}

class FlightOrder extends Order {
    - flights: List<ScheduledFlight>
    - noFlyList: List<String>
    - paymentStrategy: PaymentStrategy
    + setPaymentStrategy(PaymentStrategy): void
    + processOrder(): boolean
    - isOrderValid(...): boolean
}

interface PaymentStrategy {
    + pay(double amount): boolean
    + isValid(): boolean
}

class CreditCardPayment implements PaymentStrategy {
    - creditCard: CreditCard
    + CreditCardPayment(CreditCard)
    + pay(double amount): boolean
    + isValid(): boolean
}

class PayPalPayment implements PaymentStrategy {
    - email: String
    - password: String
    + PayPalPayment(String, String)
    + pay(double amount): boolean
    + isValid(): boolean
}

class CreditCard {
    - amount: double
    - number: String
    - date: Date
    - cvv: String
    + isValid(): boolean
    + getAmount(): double
}

```

```

    + setAmount(double): void
}

class Paypal {
    + {static} DATA_BASE: Map<String, String>
}

FlightOrder --> PaymentStrategy : delegates to
CreditCardPayment --> CreditCard : uses
PayPalPayment --> Paypal : uses

@enduml

```

Improvement: `FlightOrder` now depends only on the `PaymentStrategy` interface. Adding a new payment method (e.g., `BitcoinPayment`) only requires creating a new class that implements `PaymentStrategy` — **no changes** to `FlightOrder`.

1.7 Code Changes

PaymentStrategy.java (New Interface)

```

package flight.reservation.payment;

public interface PaymentStrategy {
    boolean pay(double amount);
    boolean isValid();
}

```

CreditCardPayment.java (New Concrete Strategy)

```

package flight.reservation.payment;

public class CreditCardPayment implements PaymentStrategy {
    private final CreditCard creditCard;

    public CreditCardPayment(CreditCard creditCard) {
        this.creditCard = creditCard;
    }

    @Override
    public boolean pay(double amount) {
        if (!isValid()) {
            return false;
        }
        System.out.println("Paying " + amount + " using Credit Card.");
        double remainingAmount = creditCard.getAmount() - amount;
        if (remainingAmount < 0) {
            System.out.printf("Card limit reached - Balance: %f\n",
                remainingAmount);
        }
    }
}

```

```

        throw new IllegalStateException("Card limit reached");
    }
    creditCard.setAmount(remainingAmount);
    return true;
}

@Override
public boolean isValid() {
    return creditCard != null && creditCard.isValid();
}
}

```

PayPalPayment.java (New Concrete Strategy)

```

package flight.reservation.payment;

public class PayPalPayment implements PaymentStrategy {
    private final String email;
    private final String password;

    public PayPalPayment(String email, String password) {
        this.email = email;
        this.password = password;
    }

    @Override
    public boolean pay(double amount) {
        if (!isValid()) {
            return false;
        }
        System.out.println("Paying " + amount + " using PayPal.");
        return true;
    }

    @Override
    public boolean isValid() {
        return email != null && password != null
            && email.equals(Paypal.DATA_BASE.get(password));
    }
}

```

FlightOrder.java (Refactored)

```

package flight.reservation.order;

import flight.reservation.Customer;
import flight.reservation.flight.ScheduledFlight;
import flight.reservation.payment.PaymentStrategy;

```

```

import java.util.Arrays;
import java.util.List;

public class FlightOrder extends Order {
    private final List<ScheduledFlight> flights;
    private PaymentStrategy paymentStrategy;
    static List<String> noFlyList = Arrays.asList("Peter", "Johannes");

    public FlightOrder(List<ScheduledFlight> flights) {
        this.flights = flights;
    }

    public static List<String> getNoFlyList() {
        return noFlyList;
    }

    public List<ScheduledFlight> getScheduledFlights() {
        return flights;
    }

    public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

    public boolean processOrder() throws IllegalStateException {
        if (isClosed()) {
            return true;
        }
        if (paymentStrategy == null || !paymentStrategy.isValid()) {
            throw new IllegalStateException("Payment information is not set or not
valid.");
        }
        boolean isPaid = paymentStrategy.pay(this.getPrice());
        if (isPaid) {
            this.setClosed();
        }
        return isPaid;
    }
}

```

Usage Example

```

// Before (tightly coupled):
order.processOrderWithCreditCard(creditCard);
order.processOrderWithPayPal("amanda@ya.com", "amanda1985");

// After (strategy-based):
order.setPaymentStrategy(new CreditCardPayment(creditCard));
order.processOrder();

// or

```

```
order.setPaymentStrategy(new PayPalPayment("amanda@ya.com", "amanda1985"));
order.processOrder();

// Easy to add new payment:
order.setPaymentStrategy(new BitcoinPayment(walletAddress));
order.processOrder();
```

2. Factory Pattern — Aircraft Creation

2.1 Problem Identification

In the current codebase, aircraft objects are created via **direct instantiation** scattered across the code:

In `Runner.java` :

```
static List<Object> aircrafts = Arrays.asList(
    new PassengerPlane("A380"),
    new PassengerPlane("A350"),
    new Helicopter("H1"),
    new PassengerDrone("HypaHype")
);
```

In `Flight.java` — `isAircraftValid()` method:

```
if (this.aircraft instanceof PassengerPlane) {
    model = ((PassengerPlane) this.aircraft).model;
} else if (this.aircraft instanceof Helicopter) {
    model = ((Helicopter) this.aircraft).getModel();
} else if (this.aircraft instanceof PassengerDrone) {
    model = "HypaHype";
} else {
    throw new IllegalArgumentException("Aircraft is not recognized");
}
```

Key issues:

- The `aircraft` field in `Flight` is typed as `Object` — no type safety.
- **instanceof chains** appear in `Flight.isAircraftValid()` , `ScheduledFlight.getCapacity()` , and `ScheduledFlight.getCrewMemberCapacity()` .
- Aircraft creation logic is **scattered** — each client must know which concrete class to instantiate.
- Adding a new aircraft type requires modifying **every instanceof chain** across multiple files.

Reference: [Factory Pattern — Refactoring Guru](#)

2.2 Pattern Description

The **Factory Pattern** is a creational design pattern that provides an interface for creating objects without specifying their exact class. A factory method or factory class encapsulates object creation, so the client code works with the product through a common interface.

Key participants:

- **Product (interface):** Defines the common interface for all aircraft.
- **Concrete Products:** `PassengerPlane` , `Helicopter` , `PassengerDrone` implement the interface.
- **Factory:** A class with a creation method that returns the appropriate aircraft based on input.

Reference: [Professor's Factory Pattern Example — CourseFactory](#)

2.3 Application and Reasoning

Why Factory Pattern fits here:

1. There are **multiple types of aircraft** (`PassengerPlane` , `Helicopter` , `PassengerDrone`) that should share a common interface.
2. Aircraft creation involves **model-specific logic** (capacity, crew, validation) that is currently scattered.
3. The `Object` type for aircraft is a clear design flaw — a factory naturally produces objects of a **known interface type**.
4. New aircraft types (e.g., `CargoPlane` , `JetFighter`) should be addable without modifying existing code.

How it is applied:

- Create an `Aircraft` interface with methods `getModel()` , `getPassengerCapacity()` , `getCrewCapacity()` .
- Make `PassengerPlane` , `Helicopter` , `PassengerDrone` implement `Aircraft` .
- Create an `AircraftFactory` with a static `createAircraft(String type, String model)` method.

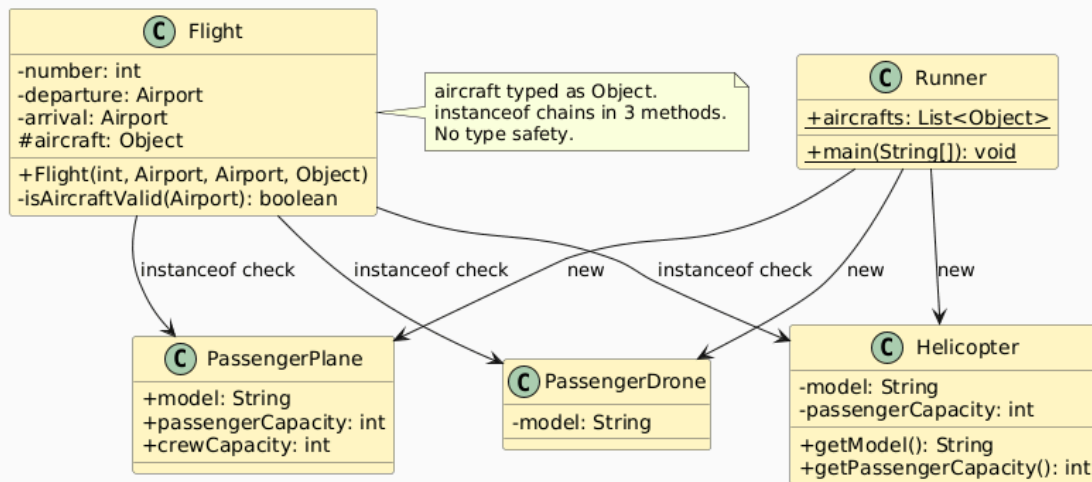
Note: This pattern works hand-in-hand with the **Adapter Pattern** (Section 3), which introduces the `Aircraft` interface. The Factory Pattern builds on top of that interface to centralize creation logic.

2.4 Benefits and Drawbacks

Benefits	Drawbacks
Centralized creation: All aircraft instantiation happens in one place.	Adds an extra layer of indirection.
Type safety: Returns <code>Aircraft</code> interface instead of <code>Object</code> .	Factory must be updated when a new aircraft type is added (but only in one place).
Eliminates scattered new calls: Clients don't need to know concrete classes.	Slightly more complex for a small number of types.
Single point of change: Adding a new aircraft type requires modifying only the factory.	

2.5 Class Diagram — Before

Before: Factory Pattern



```

@startuml
title Before Factory Pattern

class Flight {
    - number: int
    - departure: Airport
    - arrival: Airport
    # aircraft: Object
    + Flight(int, Airport, Airport, Object)
    - isAircraftValid(Airport): boolean
}

class PassengerPlane {
    + model: String
    + passengerCapacity: int
    + crewCapacity: int
}

class Helicopter {
    - model: String
    - passengerCapacity: int
    + getModel(): String
    + getPassengerCapacity(): int
}

class PassengerDrone {
    - model: String
}

class Runner {
    + {static} aircrafts: List<Object>
}
    
```



```

Flight --> PassengerPlane : instanceof check
Flight --> Helicopter : instanceof check
Flight --> PassengerDrone : instanceof check
Runner --> PassengerPlane : new
Runner --> Helicopter : new
Runner --> PassengerDrone : new

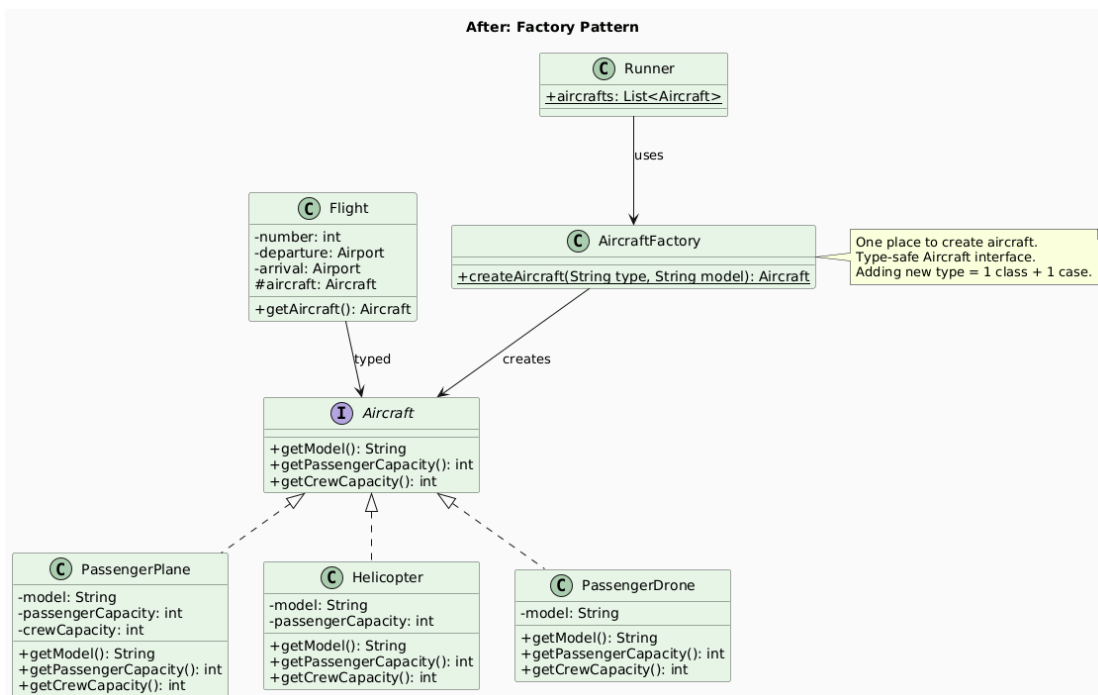
```

note right of Flight : aircraft field is typed as Object\nuses instanceof chains everywhere

@enduml

Problem: No common interface. Object typing. instanceof chains throughout.

2.6 Class Diagram — After



```

@startuml
title After Factory Pattern

interface Aircraft {
    + getModel(): String
    + getPassengerCapacity(): int
    + getCrewCapacity(): int
}

class PassengerPlane implements Aircraft {
    - model: String
    - passengerCapacity: int
}

```

```

    - crewCapacity: int
    + getModel(): String
    + getPassengerCapacity(): int
    + getCrewCapacity(): int
}

class Helicopter implements Aircraft {
    - model: String
    - passengerCapacity: int
    + getModel(): String
    + getPassengerCapacity(): int
    + getCrewCapacity(): int
}

class PassengerDrone implements Aircraft {
    - model: String
    + getModel(): String
    + getPassengerCapacity(): int
    + getCrewCapacity(): int
}

class AircraftFactory {
    + {static} createAircraft(String type, String model): Aircraft
}

class Flight {
    - number: int
    - departure: Airport
    - arrival: Airport
    # aircraft: Aircraft
    + getAircraft(): Aircraft
}

AircraftFactory --> Aircraft : creates
Flight --> Aircraft : uses interface

@enduml

```

Improvement: `Flight` now holds an `Aircraft` reference (not `Object`). All creation goes through `AircraftFactory`. No more `instanceof` chains for basic properties.

2.7 Code Changes

Note: The `Aircraft` interface shown below is formally introduced as part of the **Adapter Pattern (Section 3)**, which resolves the incompatible interfaces between aircraft classes. Both the Factory and Adapter patterns share this interface — the Adapter creates it, and the Factory produces objects typed to it. The code for all three aircraft classes is shown in full in Section 3.

Aircraft.java (New Interface)

```

package flight.reservation.plane;

public interface Aircraft {
    String getModel();
    int getPassengerCapacity();
    int getCrewCapacity();
}

```

PassengerPlane.java (Refactored)

```

package flight.reservation.plane;

public class PassengerPlane implements Aircraft {
    private String model;
    private int passengerCapacity;
    private int crewCapacity;

    public PassengerPlane(String model) {
        this.model = model;
        switch (model) {
            case "A380":
                passengerCapacity = 500;
                crewCapacity = 42;
                break;
            case "A350":
                passengerCapacity = 320;
                crewCapacity = 40;
                break;
            case "Embraer 190":
                passengerCapacity = 25;
                crewCapacity = 5;
                break;
            case "Antonov AN2":
                passengerCapacity = 15;
                crewCapacity = 3;
                break;
            default:
                throw new IllegalArgumentException(
                    String.format("Model type '%s' is not recognized", model));
        }
    }

    @Override
    public String getModel() { return model; }

    @Override
    public int getPassengerCapacity() { return passengerCapacity; }

    @Override

```

```
    public int getCrewCapacity() { return crewCapacity; }  
}
```

Helicopter.java (Refactored)

```
package flight.reservation.plane;  
  
public class Helicopter implements Aircraft {  
    private final String model;  
    private final int passengerCapacity;  
  
    public Helicopter(String model) {  
        this.model = model;  
        if (model.equals("H1")) {  
            passengerCapacity = 4;  
        } else if (model.equals("H2")) {  
            passengerCapacity = 6;  
        } else {  
            throw new IllegalArgumentException(  
                String.format("Model type '%s' is not recognized", model));  
        }  
    }  
  
    @Override  
    public String getModel() { return model; }  
  
    @Override  
    public int getPassengerCapacity() { return passengerCapacity; }  
  
    @Override  
    public int getCrewCapacity() { return 2; }  
}
```

PassengerDrone.java (Refactored)

```
package flight.reservation.plane;  
  
public class PassengerDrone implements Aircraft {  
    private final String model;  
  
    public PassengerDrone(String model) {  
        if (model.equals("HypaHype")) {  
            this.model = model;  
        } else {  
            throw new IllegalArgumentException(  
                String.format("Model type '%s' is not recognized", model));  
        }  
    }  
}
```

```

@Override
public String getModel() { return model; }

@Override
public int getPassengerCapacity() { return 4; }

@Override
public int getCrewCapacity() { return 0; }
}

```

AircraftFactory.java (New Factory Class)

```

package flight.reservation.plane;

public class AircraftFactory {

    public static Aircraft createAircraft(String type, String model) {
        switch (type.toLowerCase()) {
            case "plane":
                return new PassengerPlane(model);
            case "helicopter":
                return new Helicopter(model);
            case "drone":
                return new PassengerDrone(model);
            default:
                throw new IllegalArgumentException(
                    String.format("Aircraft type '%s' is not recognized", type));
        }
    }
}

```

Usage Example

```

// Before (scattered direct instantiation):
List<Object> aircrafts = Arrays.asList(
    new PassengerPlane("A380"),
    new Helicopter("H1"),
    new PassengerDrone("HypaHype")
);

// After (centralized factory creation):
List<Aircraft> aircrafts = Arrays.asList(
    AircraftFactory.createAircraft("plane", "A380"),
    AircraftFactory.createAircraft("helicopter", "H1"),
    AircraftFactory.createAircraft("drone", "HypaHype")
);

```

3. Adapter Pattern — Unified Aircraft Interface

3.1 Problem Identification

The three aircraft types in the codebase — `PassengerPlane` , `Helicopter` , and `PassengerDrone` — are **completely unrelated classes** with **no common interface**:

Property	PassengerPlane	Helicopter	PassengerDrone
Model access	<code>public String model</code> (field)	<code>getModel()</code> (method)	No public accessor
Capacity access	<code>public int passengerCapacity</code> (field)	<code>getPassengerCapacity()</code> (method)	Hardcoded 4 in <code>ScheduledFlight</code>
Crew capacity	<code>public int crewCapacity</code> (field)	Hardcoded 2 in <code>ScheduledFlight</code>	Hardcoded 0 in <code>ScheduledFlight</code>

This leads to **`instanceof` cascades** throughout the codebase:

In `Flight.isAircraftValid()` (lines 32-44):

```
if (this.aircraft instanceof PassengerPlane) {
    model = ((PassengerPlane) this.aircraft).model;
} else if (this.aircraft instanceof Helicopter) {
    model = ((Helicopter) this.aircraft).getModel();
} else if (this.aircraft instanceof PassengerDrone) {
    model = "HypaHype";
}
```

In `ScheduledFlight.getCapacity()` (lines 53-64):

```
if (this.aircraft instanceof PassengerPlane) {
    return ((PassengerPlane) this.aircraft).passengerCapacity;
}
if (this.aircraft instanceof Helicopter) {
    return ((Helicopter) this.aircraft).getPassengerCapacity();
}
if (this.aircraft instanceof PassengerDrone) {
    return 4;
}
```

In `ScheduledFlight.getCrewMemberCapacity()` (lines 32-43): Same pattern with hardcoded values.

This violates the **Open/Closed Principle** — adding a new aircraft type requires modifying every `instanceof` chain.

Reference: [Adapter Pattern — Refactoring Guru](#)

3.2 Pattern Description

The **Adapter Pattern** is a structural design pattern that allows objects with incompatible interfaces to collaborate. It wraps an existing class with a new interface so that it becomes compatible with the client's expected interface.

In this case, the three aircraft classes have **incompatible interfaces** (different field access patterns, different method names, missing methods). The Adapter Pattern introduces a common `Aircraft` interface and adapts each class to conform to it.

Key participants:

- **Target Interface (`Aircraft`):** The unified interface that `Flight` and `ScheduledFlight` expect.
- **Adaptees:** The original `PassengerPlane` , `Helicopter` , `PassengerDrone` classes with their inconsistent interfaces.
- **Adaptation:** Each class is refactored to implement `Aircraft` , making their incompatible interfaces uniform.

Reference: [Professor's Adapter Pattern Example — IoTAdapter](#)

3.3 Application and Reasoning

Why Adapter Pattern fits here:

1. The three aircraft classes serve the **same conceptual role** (they are all aircraft) but have **incompatible interfaces** — different access patterns for model, capacity, and crew.
2. `Flight` stores aircraft as `Object` because there's no common type — a clear sign that an interface is needed.
3. The professor's IoT Adapter example shows the exact same scenario: `NodeData` and `VideoNode` were incompatible interfaces unified via an adapter. Here, we unify `PassengerPlane` , `Helicopter` , and `PassengerDrone` under `Aircraft` .

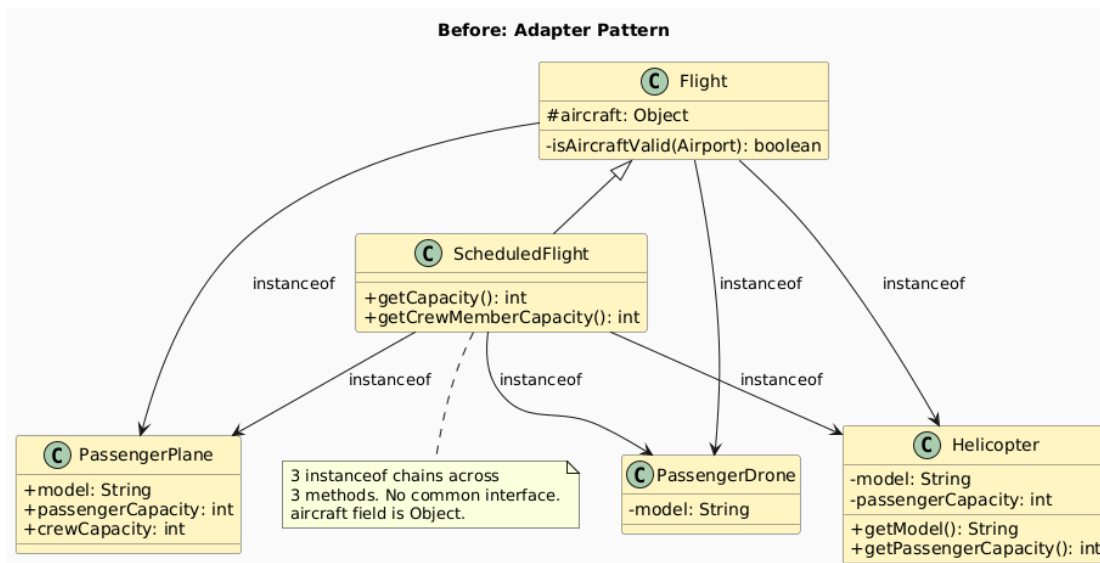
How it is applied:

- Define an `Aircraft` interface with `getModel()` , `getPassengerCapacity()` , `getCrewCapacity()` .
- Adapt each aircraft class to implement `Aircraft` (making fields private, adding getters).
- Change `Flight.aircraft` from `Object` to `Aircraft` .
- **Eliminate all `instanceof` chains** — replace with polymorphic calls.

3.4 Benefits and Drawbacks

Benefits	Drawbacks
Eliminates all <code>instanceof</code> chains across <code>Flight</code> and <code>ScheduledFlight</code> .	Requires modifying existing aircraft classes (or wrapping them).
Type safety: <code>Flight.aircraft</code> is <code>Aircraft</code> instead of <code>Object</code> .	If original classes can't be modified, wrapper adapters add an extra layer.
Polymorphism: Capacity and model are accessed uniformly via interface methods.	In this case the adaptation is straightforward, so drawbacks are minimal.
Extensibility: New aircraft types just implement <code>Aircraft</code> — no other changes needed.	

3.5 Class Diagram — Before



```
@startuml
title Before Adapter Pattern

class Flight {
    # aircraft: Object
    - isAircraftValid(Airport): boolean
}

class ScheduledFlight extends Flight {
    + getCapacity(): int
    + getCrewMemberCapacity(): int
}

class PassengerPlane {
    + model: String
    + passengerCapacity: int
    + crewCapacity: int
}

class Helicopter {
    - model: String
    - passengerCapacity: int
    + getModel(): String
    + getPassengerCapacity(): int
}

class PassengerDrone {
    - model: String
}
```



```

Flight --> PassengerPlane : instanceof
Flight --> Helicopter : instanceof
Flight --> PassengerDrone : instanceof
ScheduledFlight --> PassengerPlane : instanceof
ScheduledFlight --> Helicopter : instanceof
ScheduledFlight --> PassengerDrone : instanceof

```

```

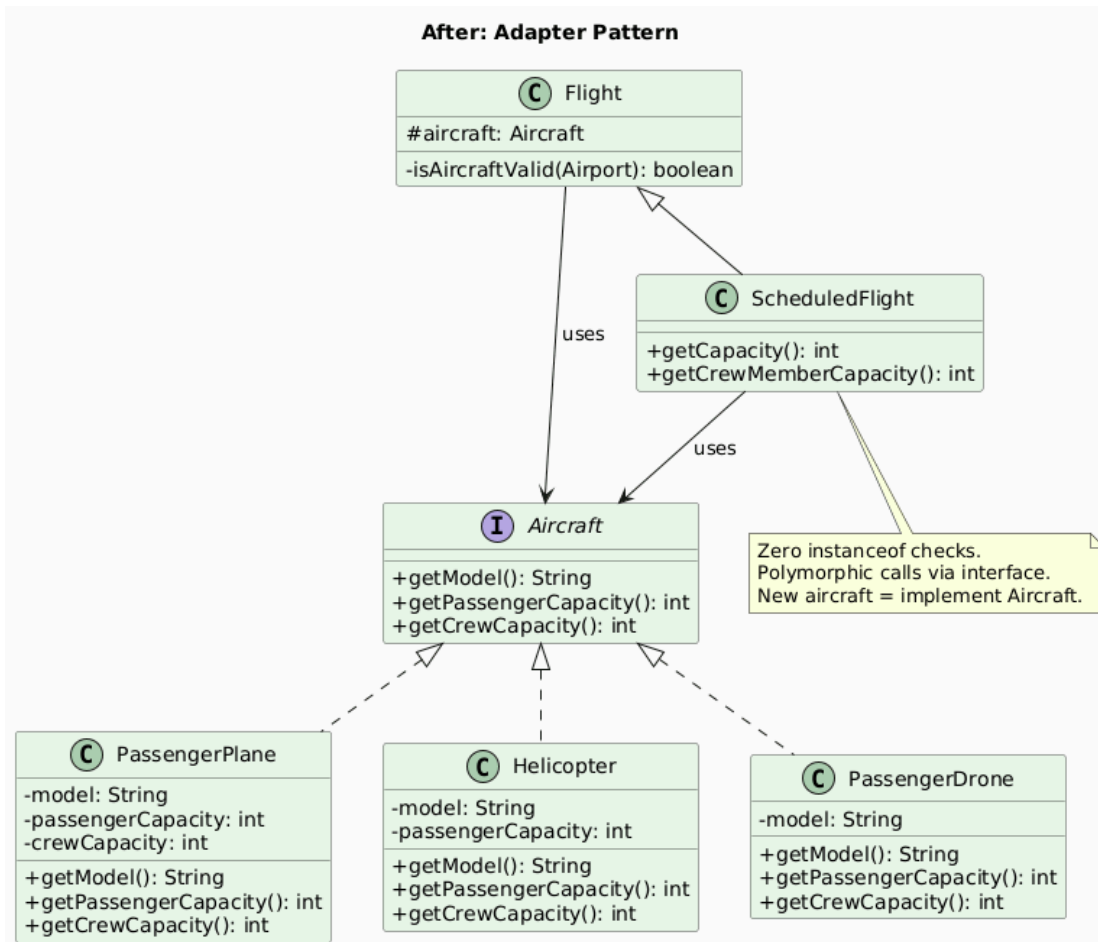
note bottom of Flight
    aircraft typed as Object
    instanceof checks everywhere
end note

```

```
@enduml
```

Problem: No shared interface. `Object` typing. 3 separate `instanceof` chains in 3 methods.

3.6 Class Diagram — After



```

@startuml
title After Adapter Pattern

```

```

interface Aircraft {
    + getModel(): String
    + getPassengerCapacity(): int
    + getCrewCapacity(): int
}

class PassengerPlane implements Aircraft {
    - model: String
    - passengerCapacity: int
    - crewCapacity: int
    + getModel(): String
    + getPassengerCapacity(): int
    + getCrewCapacity(): int
}

class Helicopter implements Aircraft {
    - model: String
    - passengerCapacity: int
    + getModel(): String
    + getPassengerCapacity(): int
    + getCrewCapacity(): int
}

class PassengerDrone implements Aircraft {
    - model: String
    + getModel(): String
    + getPassengerCapacity(): int
    + getCrewCapacity(): int
}

class Flight {
    # aircraft: Aircraft
    - isAircraftValid(Airport): boolean
}

class ScheduledFlight extends Flight {
    + getCapacity(): int
    + getCrewMemberCapacity(): int
}

Flight --> Aircraft : uses interface
ScheduledFlight --> Aircraft : uses interface

note bottom of Flight
    No more instanceof checks!
    Polymorphic calls via Aircraft interface
end note

@enduml

```

Improvement: All aircraft types implement `Aircraft`. `Flight` and `ScheduledFlight` use only the interface — all `instanceof` chains are eliminated.

3.7 Code Changes

Flight.java (Refactored — instanceof chains removed)

```
package flight.reservation.flight;

import flight.reservation.Airport;
import flight.reservation.plane.Aircraft;

import java.util.Arrays;

public class Flight {

    private int number;
    private Airport departure;
    private Airport arrival;
    protected Aircraft aircraft;

    public Flight(int number, Airport departure, Airport arrival, Aircraft aircraft)
        throws IllegalArgumentException {
        this.number = number;
        this.departure = departure;
        this.arrival = arrival;
        this.aircraft = aircraft;
        checkValidity();
    }

    private void checkValidity() throws IllegalArgumentException {
        if (!isAircraftValid(departure) || !isAircraftValid(arrival)) {
            throw new IllegalArgumentException(
                "Selected aircraft is not valid for the selected route.");
        }
    }

    private boolean isAircraftValid(Airport airport) {
        // No more instanceof! Just use the Aircraft interface
        String model = aircraft.getModel();
        return Arrays.stream(airport.getAllowedAircrafts())
            .anyMatch(allowed -> allowed.equals(model));
    }

    public Aircraft getAircraft() {
        return aircraft;
    }

    public int getNumber() { return number; }
    public Airport getDeparture() { return departure; }
    public Airport getArrival() { return arrival; }
```

```

@Override
public String toString() {
    return aircraft.getModel() + "-" + number + "-"
        + departure.getCode() + "/" + arrival.getCode();
}
}

```

ScheduledFlight.java (Refactored — instanceof chains removed)

```

package flight.reservation.flight;

import flight.reservation.Airport;
import flight.reservation.Passenger;
import flight.reservation.plane.Aircraft;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class ScheduledFlight extends Flight {

    private final List<Passenger> passengers;
    private final Date departureTime;
    private double currentPrice = 100;

    public ScheduledFlight(int number, Airport departure, Airport arrival,
        Aircraft aircraft, Date departureTime) {
        super(number, departure, arrival, aircraft);
        this.departureTime = departureTime;
        this.passengers = new ArrayList<>();
    }

    // No more instanceof chains! Direct interface calls.
    public int getCrewMemberCapacity() {
        return aircraft.getCrewCapacity();
    }

    public int getCapacity() {
        return aircraft.getPassengerCapacity();
    }

    public int getAvailableCapacity() {
        return this.getCapacity() - this.passengers.size();
    }

    public void addPassengers(List<Passenger> passengers) {
        this.passengers.addAll(passengers);
    }

    public void removePassengers(List<Passenger> passengers) {

```

```

        this.passengers.removeAll(passengers);
    }

    public Date getDepartureTime() { return departureTime; }
    public List<Passenger> getPassengers() { return passengers; }
    public double getCurrentPrice() { return currentPrice; }
    public void setCurrentPrice(double currentPrice) { this.currentPrice =
currentPrice; }
}

```

Impact Summary

File	Before	After
Flight.isAircraftValid()	12-line instanceof chain	2-line polymorphic call
ScheduledFlight.getCapacity()	11-line instanceof chain	1-line: return aircraft.getPassengerCapacity()
ScheduledFlight.getCrewMemberCapacity()	10-line instanceof chain	1-line: return aircraft.getCrewCapacity()
Flight.aircraft type	Object	Aircraft

Side-effect: As a positive consequence of this refactoring, the `throws NoSuchFieldException` declarations previously required on `getCapacity()`, `getAvailableCapacity()`, and `getCrewMemberCapacity()` are fully eliminated. These checked exceptions existed solely because the `instanceof` branches could fall through without a match. With a proper interface, this failure mode is impossible at compile time. `Customer.isOrderValid()` — which called these methods — is also simplified as a result.

4. Observer Pattern — Flight Change Notifications

4.1 Problem Identification

The current `ScheduledFlight` and `Schedule` classes have **no notification mechanism**. When a flight's price changes, capacity fills up, or a flight is cancelled, no part of the system is informed. Specifically:

- `ScheduledFlight.setCurrentPrice(double)` silently mutates price — no one is notified.
- `ScheduledFlight.addPassengers()` and `removePassengers()` change capacity — no observer is informed.
- `Schedule.removeFlight()` cancels a scheduled flight — no booked customer receives a notification.
- `Customer` has a list of `orders`, each linked to `ScheduledFlight` s, but no callback mechanism exists.

This means customers cannot react to flight changes, and the system cannot support features like price drop alerts, overbooking warnings, or cancellation notifications — all critical for a real reservation system.

Reference: [Observer Pattern — Refactoring Guru](#)

4.2 Pattern Description

The **Observer Pattern** is a behavioral design pattern where an object (the **Subject**) maintains a list of dependents (**Observers**) and automatically notifies them when its state changes. Observers register themselves with the subject and receive updates via a callback method.

Key participants:

- **Observer (interface):** Declares the `update()` callback method all observers must implement.
- **Subject (`ScheduledFlight`):** Maintains a list of observers and calls `notify()` on state changes.
- **Concrete Observer (`Customer`):** Implements the `FlightObserver` interface and reacts to notifications.

Reference: [Professor's Design Patterns Repository](#)

4.3 Application and Reasoning

Why Observer Pattern fits here:

1. `ScheduledFlight` is a natural **Subject** — its state (price, passenger count, cancellation) changes over time and multiple parties may be interested.
2. `Customer` is a natural **Observer** — they have booked seats on flights and should be informed of relevant changes.
3. The pattern allows **loose coupling**: `ScheduledFlight` does not need to know the concrete type of who is watching — it just notifies `FlightObserver` references.
4. New observer types (e.g., a `NotificationService`, `AdminDashboard`) can be added without changing `ScheduledFlight`.

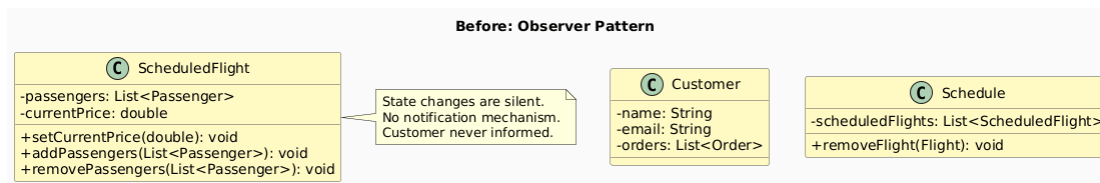
How it is applied:

- Create a `FlightObserver` interface with `update(String event, ScheduledFlight flight)`.
- Add observer registration/removal methods to `ScheduledFlight`.
- Trigger notifications in `setCurrentPrice()`, `addPassengers()`, and `removePassengers()`.
- `Customer` implements `FlightObserver` and registers itself when booking.

4.4 Benefits and Drawbacks

Benefits	Drawbacks
Loose coupling: <code>ScheduledFlight</code> has no direct dependency on <code>Customer</code> .	Unexpected or cascading updates if observer logic is complex.
Extensibility: New observers (dashboard, email service) require zero changes to <code>ScheduledFlight</code> .	Observers must be explicitly deregistered to avoid memory leaks.
Real-time responsiveness: Customers are immediately notified of price changes or cancellations.	Order of notification is not guaranteed if multiple observers are registered.
Single Responsibility: Notification concern is separate from flight management logic.	

4.5 Class Diagram — Before



```
@startuml
title Before Observer Pattern

class ScheduledFlight {
    - passengers: List<Passenger>
    - currentPrice: double
    + setCurrentPrice(double): void
    + addPassengers(List<Passenger>): void
    + removePassengers(List<Passenger>): void
}

class Customer {
    - name: String
    - email: String
    - orders: List<Order>
}

class Schedule {
    - scheduledFlights: List<ScheduledFlight>
    + removeFlight(Flight): void
}

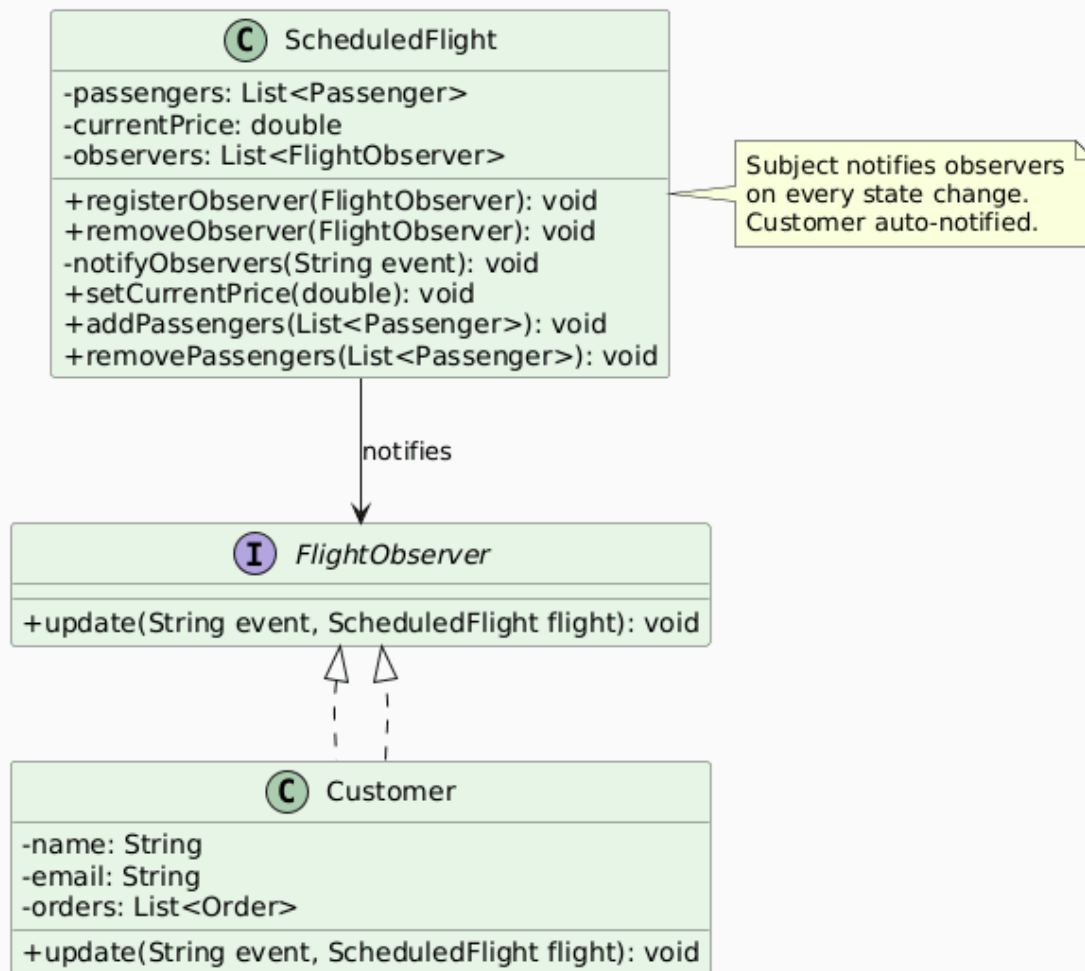
note right of ScheduledFlight
    Price and capacity changes
    are silent – no notification
    mechanism exists
end note

@enduml
```

Problem: State changes in `ScheduledFlight` are invisible to `Customer` and the rest of the system.

4.6 Class Diagram — After

After: Observer Pattern



```

@startuml
title After Observer Pattern

interface FlightObserver {
    + update(String event, ScheduledFlight flight): void
}

class ScheduledFlight {
    - passengers: List<Passenger>
    - currentPrice: double
    - observers: List<FlightObserver>
    + registerObserver(FlightObserver): void
    + removeObserver(FlightObserver): void
    - notifyObservers(String event): void
    + setCurrentPrice(double): void
    + addPassengers(List<Passenger>): void
    + removePassengers(List<Passenger>): void
}

Customer ..|> FlightObserver
ScheduledFlight --> FlightObserver : notifies
    
```



```

}

class Customer implements FlightObserver {
    - name: String
    - email: String
    - orders: List<Order>
    + update(String event, ScheduledFlight flight): void
    + createOrder(...): FlightOrder
}

ScheduledFlight --> FlightObserver : notifies
Customer ..|> FlightObserver : implements

@enduml

```

Improvement: `ScheduledFlight` notifies all registered `FlightObserver`s on state changes. `Customer` implements the interface and receives notifications. New observer types plug in without any changes to `ScheduledFlight`.

4.7 Code Changes

FlightObserver.java (New Interface)

```

package flight.reservation.flight;

public interface FlightObserver {
    void update(String event, ScheduledFlight flight);
}

```

ScheduledFlight.java (Refactored — Observer support added)

```

package flight.reservation.flight;

import flight.reservation.Airport;
import flight.reservation.Passenger;
import flight.reservation.plane.Aircraft;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class ScheduledFlight extends Flight {

    private final List<Passenger> passengers;
    private final Date departureTime;
    private double currentPrice = 100;
    private final List<FlightObserver> observers = new ArrayList<>();

    public ScheduledFlight(int number, Airport departure, Airport arrival,

```

```

        Aircraft aircraft, Date departureTime) {
    super(number, departure, arrival, aircraft);
    this.departureTime = departureTime;
    this.passengers = new ArrayList<>();
}

// Observer registration
public void registerObserver(FlightObserver observer) {
    observers.add(observer);
}

public void removeObserver(FlightObserver observer) {
    observers.remove(observer);
}

private void notifyObservers(String event) {
    for (FlightObserver observer : observers) {
        observer.update(event, this);
    }
}

// State-changing methods now trigger notifications
public void setCurrentPrice(double currentPrice) {
    this.currentPrice = currentPrice;
    notifyObservers("PRICE_CHANGED");
}

public void addPassengers(List<Passenger> passengers) {
    this.passengers.addAll(passengers);
    notifyObservers("PASSENGERS_ADDED");
}

public void removePassengers(List<Passenger> passengers) {
    this.passengers.removeAll(passengers);
    notifyObservers("PASSENGERS_REMOVED");
}

public int getCrewMemberCapacity() {
    return aircraft.getCrewCapacity();
}

public int getCapacity() {
    return aircraft.getPassengerCapacity();
}

public int getAvailableCapacity() {
    return this.getCapacity() - this.passengers.size();
}

public Date getDepartureTime() { return departureTime; }
public List<Passenger> getPassengers() { return passengers; }

```

```

    public double getCurrentPrice() { return currentPrice; }
}

```

Customer.java (Refactored — implements FlightObserver)

```

package flight.reservation;

import flight.reservation.flight.FlightObserver;
import flight.reservation.flight.ScheduledFlight;
import flight.reservation.order.FlightOrder;
import flight.reservation.order.Order;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class Customer implements FlightObserver {

    private String email;
    private String name;
    private List<Order> orders;

    public Customer(String name, String email) {
        this.name = name;
        this.email = email;
        this.orders = new ArrayList<>();
    }

    @Override
    public void update(String event, ScheduledFlight flight) {
        switch (event) {
            case "PRICE_CHANGED":
                System.out.printf("[Notification] %s: Price changed on flight %d –  
new price: %.2f%n",
                                name, flight.getNumber(), flight.getCurrentPrice());
                break;
            case "PASSENGERS_ADDED":
                System.out.printf("[Notification] %s: Flight %d capacity updated –  
seats left: %d%n",
                                name, flight.getNumber(), flight.getAvailableCapacity());
                break;
            case "PASSENGERS_REMOVED":
                System.out.printf("[Notification] %s: Seats freed on flight %d –  
seats available: %d%n",
                                name, flight.getNumber(), flight.getAvailableCapacity());
                break;
        }
    }

    public FlightOrder createOrder(List<String> passengerNames,
                                   List<ScheduledFlight> flights, double price) {

```

```

        if (!isOrderValid(passengerNames, flights)) {
            throw new IllegalStateException("Order is not valid");
        }
        FlightOrder order = new FlightOrder(flights);
        order.setCustomer(this);
        order.setPrice(price);
        List<Passenger> passengers = passengerNames.stream()
            .map(Passenger::new)
            .collect(Collectors.toList());
        order.setPassengers(passengers);
        // Register as observer on each booked flight
        flights.forEach(f -> f.registerObserver(this));
        order.getScheduledFlights().forEach(sf -> sf.addPassengers(passengers));
        orders.add(order);
        return order;
    }

    private boolean isOrderValid(List<String> passengerNames, List<ScheduledFlight>
flights) {
        boolean valid = !FlightOrder.getNoFlyList().contains(this.getName());
        valid = valid && passengerNames.stream().noneMatch(p ->
FlightOrder.getNoFlyList().contains(p));
        valid = valid && flights.stream().allMatch(sf -> sf.getAvailableCapacity()
>= passengerNames.size());
        return valid;
    }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public List<Order> getOrders() { return orders; }
    public void setOrders(List<Order> orders) { this.orders = orders; }
}

```

Usage Example

```

// Customer registers as observer automatically when booking
Customer customer = new Customer("Amanda", "amanda@ya.com");
FlightOrder order = customer.createOrder(
    Arrays.asList("Amanda", "Max"), Arrays.asList(scheduledFlight), 180
);

// Any price change now notifies Amanda automatically:
scheduledFlight.setCurrentPrice(250.0);
// Output: [Notification] Amanda: Price changed on flight 1 – new price: 250.00

```

5. Builder Pattern — FlightOrder Construction

5.1 Problem Identification

`Customer.createOrder()` in `Customer.java` (lines 23–38) is a **God Method** that does far too much in a single operation:

```
public FlightOrder createOrder(List<String> passengerNames,
                               List<ScheduledFlight> flights, double price) {
    if (!isOrderValid(passengerNames, flights)) { // 1. Validate
        throw new IllegalStateException("Order is not valid");
    }
    FlightOrder order = new FlightOrder(flights); // 2. Instantiate
    order.setCustomer(this); // 3. Set customer
    order.setPrice(price); // 4. Set price
    List<Passenger> passengers = passengerNames // 5. Create passengers
        .stream().map(Passenger::new)
        .collect(Collectors.toList());
    order.setPassengers(passengers); // 6. Assign passengers to
order
    order.getScheduledFlights().forEach(sf -> // 7. Wire passengers to
flights
        sf.addPassengers(passengers));
    orders.add(order); // 8. Add to customer's list
    return order;
}
```

Issues:

- **8 distinct responsibilities** packed into one method.
- `FlightOrder` is assembled via many `set*()` calls scattered across `Customer` — the object is in an **invalid intermediate state** between construction steps.
- Adding new fields to `FlightOrder` (e.g., `seatClass`, `insurance`) requires modifying `createOrder()` directly.
- Construction steps are not reusable — if another class needs to build a `FlightOrder`, the logic must be duplicated.

Reference: [Builder Pattern — Refactoring Guru](#)

5.2 Pattern Description

The **Builder Pattern** is a creational design pattern that separates the construction of a complex object from its representation. A dedicated Builder class assembles the object step by step, and the final product is only returned when fully configured.

Key participants:

- **Builder (`FlightOrderBuilder`)**: Provides step-by-step methods for configuring the order. Returns `this` for chaining.
- **Product (`FlightOrder`)**: The complex object being assembled.
- **Director (optional)**: An orchestrator that calls builder steps in a fixed order. Here, `Customer` acts as the director.

Reference: [Professor's Builder Pattern Example — StudentRecordBuilder](#)

5.3 Application and Reasoning

Why Builder Pattern fits here:

1. `FlightOrder` has **multiple optional and mandatory fields** that must be set before the order is usable — a constructor with 5 parameters would be unwieldy.
2. The current `createOrder()` method is the textbook case the Builder Pattern is designed to solve: step-by-step object assembly.
3. Following the professor's `StudentRecordBuilder` example — `Student` (the complex product) was built step by step via `StudentBuilder`. Here, `FlightOrder` (the complex product) is built step by step via `FlightOrderBuilder`.
4. A builder makes the construction process **readable** and **self-documenting** via fluent chaining.

How it is applied:

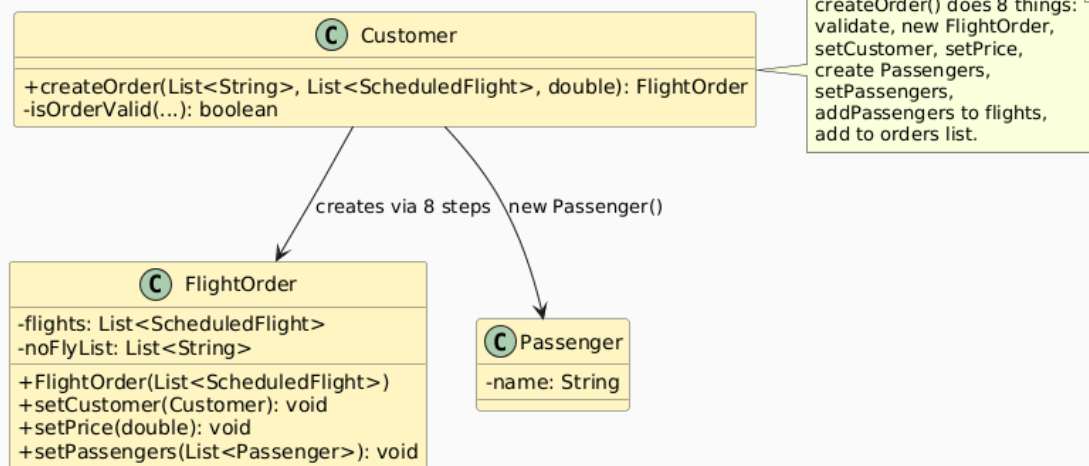
- Create `FlightOrderBuilder` with fluent methods: `forFlights()`, `withCustomer()`, `withPassengerNames()`, `withPrice()`.
- `build()` performs validation, assembles the `FlightOrder`, and returns the completed object.
- `Customer.createOrder()` delegates entirely to the builder.

5.4 Benefits and Drawbacks

Benefits	Drawbacks
Readable construction: new <code>FlightOrderBuilder().forFlights(f).withCustomer(c).withPrice(p).build()</code> is self-explanatory.	Adds a new class (<code>FlightOrderBuilder</code>).
Prevents invalid state: The product is only returned by <code>build()</code> when fully configured.	For very simple objects, a builder is overkill.
Extensibility: New fields (e.g., <code>seatClass</code>) just add a new builder method — <code>Customer.createOrder()</code> is unchanged.	
Reusability: Any class can build a <code>FlightOrder</code> using the builder without duplicating logic.	

5.5 Class Diagram — Before

Before: Builder Pattern



```

@startuml
title Before Builder Pattern

class Customer {
    + createOrder(List<String>, List<ScheduledFlight>, double): FlightOrder
    - isOrderValid(...): boolean
}

class FlightOrder {
    - flights: List<ScheduledFlight>
    - noFlyList: List<String>
    + FlightOrder(List<ScheduledFlight>)
    + setCustomer(Customer): void
    + setPrice(double): void
    + setPassengers(List<Passenger>): void
}

class Passenger {
    - name: String
}
    
```

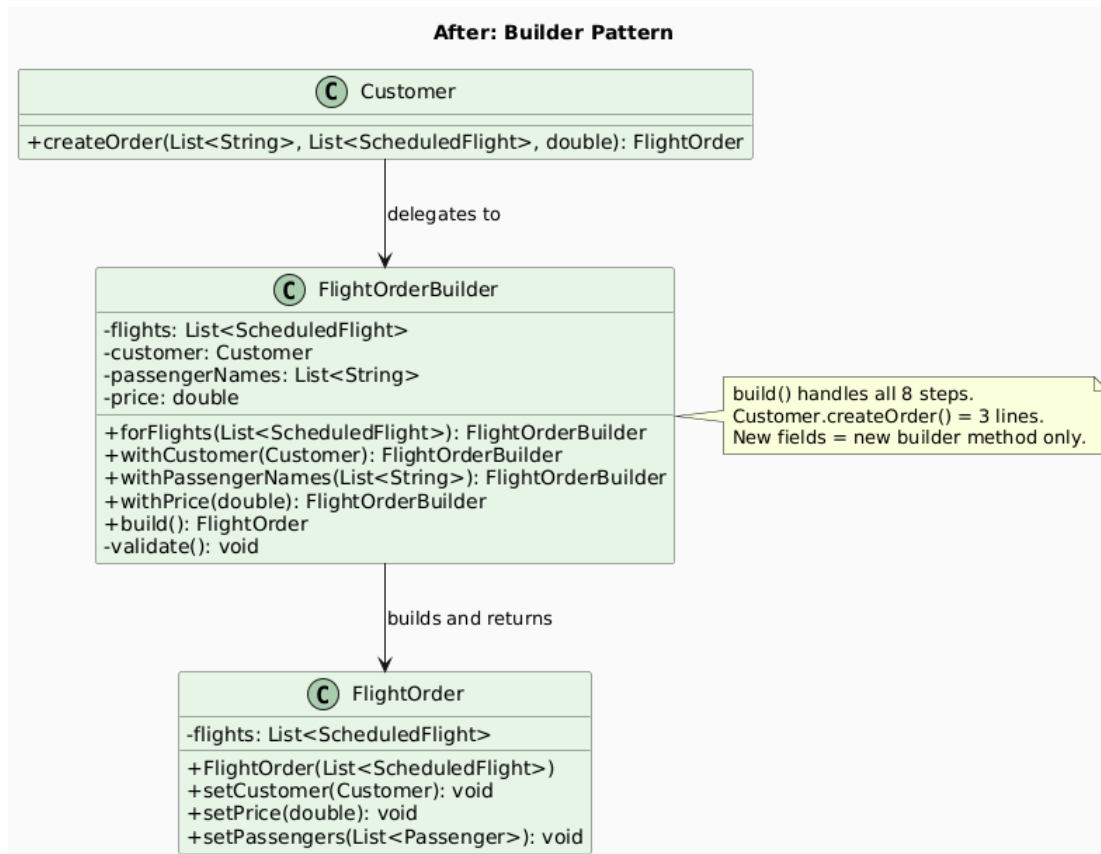
Customer --> FlightOrder : creates and sets\nfields individually
 Customer --> Passenger : creates via stream

```

note right of Customer
    createOrder() does 8 things:
    validate, new FlightOrder,
    setCustomer, setPrice,
    new Passenger[], setPassengers,
    addPassengers to flights,
    add to orders list
end note
    
```

@endum1

5.6 Class Diagram — After



```
@startuml
title After Builder Pattern

class FlightOrderBuilder {
    - flights: List<ScheduledFlight>
    - customer: Customer
    - passengerNames: List<String>
    - price: double
    + forFlights(List<ScheduledFlight>): FlightOrderBuilder
    + withCustomer(Customer): FlightOrderBuilder
    + withPassengerNames(List<String>): FlightOrderBuilder
    + withPrice(double): FlightOrderBuilder
    + build(): FlightOrder
    - validate(): void
}

class FlightOrder {
    - flights: List<ScheduledFlight>
}
```



```

+ FlightOrder(List<ScheduledFlight>)
+ setCustomer(Customer): void
+ setPrice(double): void
+ setPassengers(List<Passenger>): void
}

class Customer {
+ createOrder(List<String>, List<ScheduledFlight>, double): FlightOrder
}

Customer --> FlightOrderBuilder : uses
FlightOrderBuilder --> FlightOrder : builds and returns

@enduml

```

Improvement: `Customer.createOrder()` is now a 1-line delegation. All construction logic is encapsulated in `FlightOrderBuilder.build()`. New fields are added to the builder without touching `Customer`.

5.7 Code Changes

FlightOrderBuilder.java (New Builder Class)

```

package flight.reservation.order;

import flight.reservation.Customer;
import flight.reservation.Passenger;
import flight.reservation.flight.ScheduledFlight;

import java.util.List;
import java.util.stream.Collectors;

public class FlightOrderBuilder {

    private List<ScheduledFlight> flights;
    private Customer customer;
    private List<String> passengerNames;
    private double price;

    public FlightOrderBuilder forFlights(List<ScheduledFlight> flights) {
        this.flights = flights;
        return this;
    }

    public FlightOrderBuilder withCustomer(Customer customer) {
        this.customer = customer;
        return this;
    }

    public FlightOrderBuilder withPassengerNames(List<String> passengerNames) {
        this.passengerNames = passengerNames;
    }
}

```

```

        return this;
    }

    public FlightOrderBuilder withPrice(double price) {
        this.price = price;
        return this;
    }

    public FlightOrder build() {
        validate();

        FlightOrder order = new FlightOrder(flights);
        order.setCustomer(customer);
        order.setPrice(price);

        List<Passenger> passengers = passengerNames.stream()
            .map(Passenger::new)
            .collect(Collectors.toList());
        order.setPassengers(passengers);
        order.getScheduledFlights().forEach(sf -> sf.addPassengers(passengers));

        return order;
    }

    private void validate() {
        if (flights == null || flights.isEmpty()) {
            throw new IllegalStateException("Flights must be specified.");
        }
        if (customer == null) {
            throw new IllegalStateException("Customer must be specified.");
        }
        if (passengerNames == null || passengerNames.isEmpty()) {
            throw new IllegalStateException("Passenger names must be specified.");
        }
        if (FlightOrder.getNoFlyList().contains(customer.getName())) {
            throw new IllegalStateException("Customer is on the no-fly list.");
        }
        boolean anyPassengerOnNoFlyList = passengerNames.stream()
            .anyMatch(name -> FlightOrder.getNoFlyList().contains(name));
        if (anyPassengerOnNoFlyList) {
            throw new IllegalStateException("One or more passengers are on the no-fly list.");
        }
        boolean capacityOk = flights.stream()
            .allMatch(sf -> sf.getAvailableCapacity() >= passengerNames.size());
        if (!capacityOk) {
            throw new IllegalStateException("Insufficient capacity on one or more flights.");
        }
    }
}

```

Customer.java (Refactored — createOrder delegates to builder)

```
public FlightOrder createOrder(List<String> passengerNames,
                               List<ScheduledFlight> flights, double price) {
    FlightOrder order = new FlightOrderBuilder()
        .forFlights(flights)
        .withCustomer(this)
        .withPassengerNames(passengerNames)
        .withPrice(price)
        .build();
    orders.add(order);
    return order;
}
```

Usage Example

```
// Before (8-step God Method inside Customer.createOrder):
FlightOrder order = customer.createOrder(
    Arrays.asList("Amanda", "Max"), Arrays.asList(scheduledFlight), 180
);

// After (builder chain — each step is self-documenting):
FlightOrder order = new FlightOrderBuilder()
    .forFlights(Arrays.asList(scheduledFlight))
    .withCustomer(customer)
    .withPassengerNames(Arrays.asList("Amanda", "Max"))
    .withPrice(180)
    .build();
orders.add(order);
```

6. Chain of Responsibility Pattern — Order Validation

6.1 Problem Identification

Order validation logic is **duplicated** across two classes and **mixes multiple concerns** in a single method block.

In **Customer.java** (lines 40–53):

```
private boolean isOrderValid(List<String> passengerNames, List<ScheduledFlight>
    flights) {
    boolean valid = true;
    valid = valid && !FlightOrder.getNoFlyList().contains(this.getName());
    valid = valid && passengerNames.stream().noneMatch(
        passenger -> FlightOrder.getNoFlyList().contains(passenger));
    valid = valid && flights.stream().allMatch(scheduledFlight -> {
        try {
            return scheduledFlight.getAvailableCapacity() >= passengerNames.size();
        }
    });
}
```

```

        } catch (NoSuchFieldException e) {
            e.printStackTrace();
            return false;
        }
    });
    return valid;
}

```

In `FlightOrder.java` (lines 28–41): **Identical logic**, duplicated verbatim.

Issues:

- **DRY violation:** The same validation is written twice — a bug fix or new rule must be applied in two places.
- **Mixed concerns:** No-fly list check and capacity check are bundled together with no separation.
- **Closed to extension:** Adding a new validation rule (e.g., minimum passenger age, embargo check) requires editing the existing method in both classes.
- **Hard to test:** Individual validation rules cannot be tested in isolation.

Reference: [Chain of Responsibility — Refactoring Guru](#)

6.2 Pattern Description

The **Chain of Responsibility Pattern** is a behavioral design pattern where a request is passed along a chain of handlers. Each handler decides whether to process the request or pass it to the next handler in the chain. This decouples the sender of the request from the receiver.

Key participants:

- **Handler (abstract class):** Declares the interface for handling requests and holds a reference to the next handler.
- **Concrete Handlers:** Each implements one specific validation rule.
- **Client:** Constructs the chain and sends the request through it.

Reference: [Professor's Chain of Responsibility Example — TA-ApprovalChainOfResponsibility](#)

6.3 Application and Reasoning

Why Chain of Responsibility fits here:

1. Order validation is a sequence of **independent checks** — no-fly list, then capacity — exactly the structure the pattern is designed for.
2. Each check is a discrete responsibility; separating them matches the professor's TA approval chain (Instructor → Advisor → TACHair → AcademicOffice), where each handler validates one aspect.
3. The **duplication** between `Customer` and `FlightOrder` is eliminated — the chain is defined once and shared.
4. New validation rules (e.g., blacklisted routes, age restrictions) are added as new handler classes without touching existing code.

How it is applied:

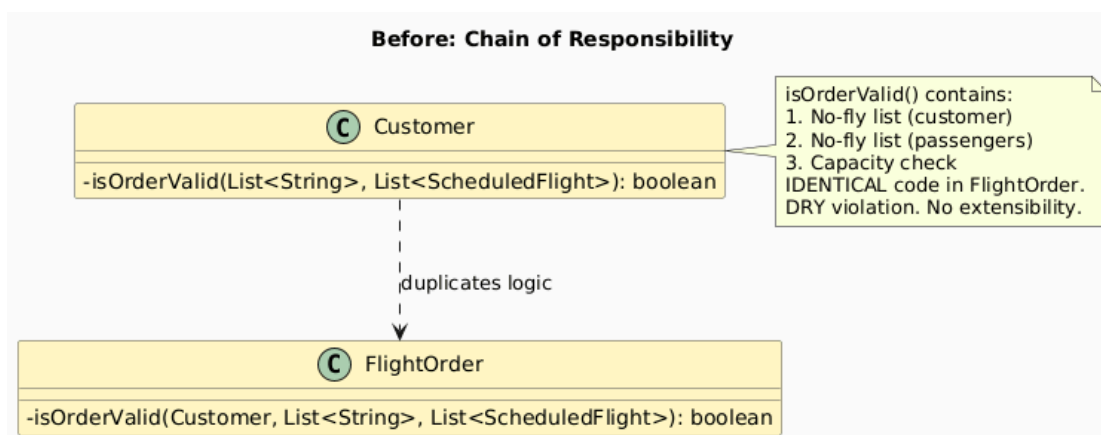
- Abstract class `OrderValidationHandler` with `setNext()` and `validate()`.
- `NoFlyListValidator` — checks customer and all passenger names against the no-fly list.
- `CapacityValidator` — checks all scheduled flights have available capacity.

- Chain: `NoFlyListValidator` → `CapacityValidator` .
- Both `Customer` and `FlightOrder` replace their `isOrderValid()` with a shared chain call.

6.4 Benefits and Drawbacks

Benefits	Drawbacks
Eliminates duplication: Single chain definition used by both <code>Customer</code> and <code>FlightOrder</code> .	Chain must be wired correctly — misconfiguration means a rule is silently skipped.
Open/Closed Principle: New validation rules = new handler class, no existing code modified.	More classes introduced (one per rule).
Single Responsibility: Each handler validates exactly one concern.	For very few rules, a simple method may be clearer.
Testability: Each validator can be unit-tested independently.	

6.5 Class Diagram — Before



```

@startuml
title Before Chain of Responsibility

class Customer {
    - isOrderValid(List<String>, List<ScheduledFlight>): boolean
}

class FlightOrder {
    - isOrderValid(Customer, List<String>, List<ScheduledFlight>): boolean
}

note right of Customer
    isOrderValid() contains:
    - no-fly list check (customer)
  
```

```

- no-fly list check (passengers)
- capacity check (all flights)
  DUPLICATED in FlightOrder
end note

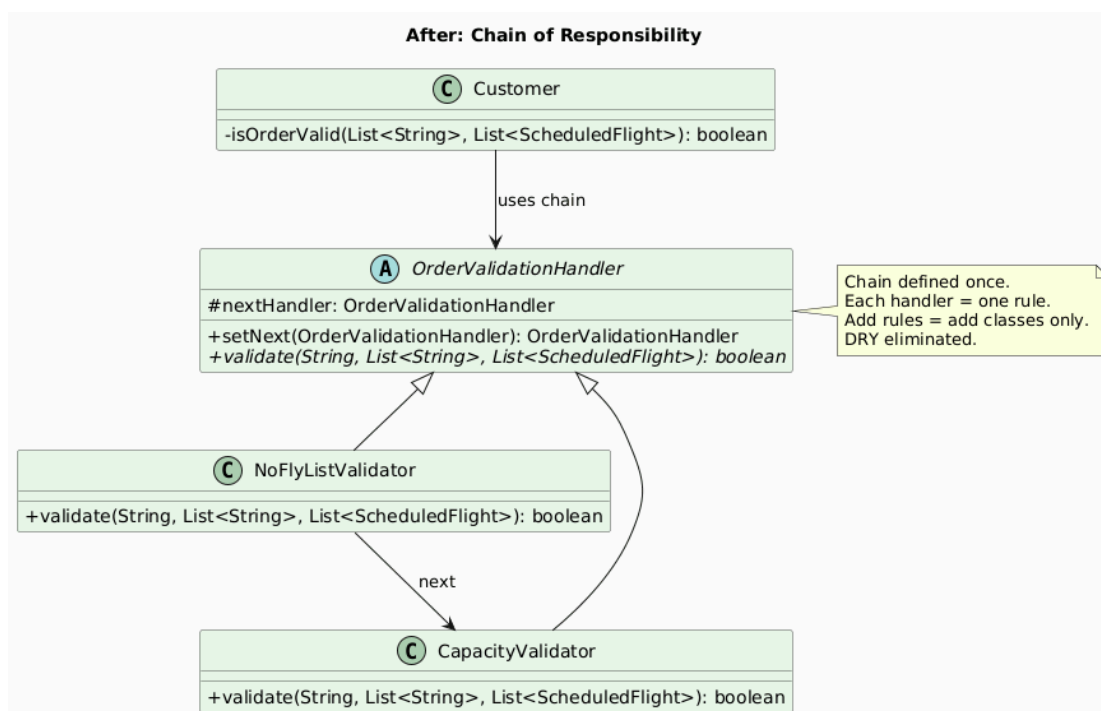
Customer ..> FlightOrder : duplicates logic

@enduml

```

Problem: Identical validation logic duplicated in two classes. Mixed concerns. No extensibility.

6.6 Class Diagram — After



```

@startuml
title After Chain of Responsibility

abstract class OrderValidationHandler {
    # nextHandler: OrderValidationHandler
    + setNext(OrderValidationHandler): OrderValidationHandler
    + {abstract} validate(String customerName, List<String> passengerNames,
List<ScheduledFlight> flights): boolean
}

class NoFlyListValidator extends OrderValidationHandler {
    + validate(String, List<String>, List<ScheduledFlight>): boolean
}

```

```

class CapacityValidator extends OrderValidationHandler {
    + validate(String, List<String>, List<ScheduledFlight>): boolean
}

class Customer {
    - buildValidationChain(): OrderValidationHandler
    + createOrder(...): FlightOrder
}

Customer --> OrderValidationHandler : uses chain
NoFlyListValidator --> CapacityValidator : next in chain

@enduml

```

Improvement: Validation is a chain. Each handler has exactly one responsibility. `Customer` uses the chain; new rules are added as new classes.

6.7 Code Changes

OrderValidationHandler.java (New Abstract Handler)

```

package flight.reservation.validation;

import flight.reservation.flight.ScheduledFlight;

import java.util.List;

public abstract class OrderValidationHandler {

    protected OrderValidationHandler nextHandler;

    public OrderValidationHandler setNext(OrderValidationHandler nextHandler) {
        this.nextHandler = nextHandler;
        return nextHandler;
    }

    public abstract boolean validate(String customerName,
                                     List<String> passengerNames,
                                     List<ScheduledFlight> flights);
}

```

NoFlyListValidator.java (New Concrete Handler)

```

package flight.reservation.validation;

import flight.reservation.flight.ScheduledFlight;
import flight.reservation.order.FlightOrder;

import java.util.List;

```

```

public class NoFlyListValidator extends OrderValidationHandler {

    @Override
    public boolean validate(String customerName,
                           List<String> passengerNames,
                           List<ScheduledFlight> flights) {
        if (FlightOrder.getNoFlyList().contains(customerName)) {
            return false;
        }
        boolean passengerClear = passengerNames.stream()
            .noneMatch(name -> FlightOrder.getNoFlyList().contains(name));
        if (!passengerClear) {
            return false;
        }
        // Pass to next handler if present
        if (nextHandler != null) {
            return nextHandler.validate(customerName, passengerNames, flights);
        }
        return true;
    }
}

```

CapacityValidator.java (New Concrete Handler)

```

package flight.reservation.validation;

import flight.reservation.flight.ScheduledFlight;

import java.util.List;

public class CapacityValidator extends OrderValidationHandler {

    @Override
    public boolean validate(String customerName,
                           List<String> passengerNames,
                           List<ScheduledFlight> flights) {
        boolean capacityOk = flights.stream()
            .allMatch(sf -> sf.getAvailableCapacity() >= passengerNames.size());
        if (!capacityOk) {
            return false;
        }
        if (nextHandler != null) {
            return nextHandler.validate(customerName, passengerNames, flights);
        }
        return true;
    }
}

```

Customer.java (Refactored — uses validation chain)


```
import flight.reservation.validation.CapacityValidator;
import flight.reservation.validation.NoFlyListValidator;
import flight.reservation.validation.OrderValidationHandler;

// Inside Customer class:
private boolean isOrderValid(List<String> passengerNames, List<ScheduledFlight>
flights) {
    OrderValidationHandler chain = new NoFlyListValidator();
    chain.setNext(new CapacityValidator());
    return chain.validate(this.getName(), passengerNames, flights);
}
```

Usage Example

```
// Before (duplicated in 2 classes, mixed concerns):
valid = valid && !FlightOrder.getNoFlyList().contains(this.getName());
valid = valid && passengerNames.stream().noneMatch(p -> noFlyList.contains(p));
valid = valid && flights.stream().allMatch(sf -> sf.getAvailableCapacity() >= ...);

// After (chain – defined once, extensible):
OrderValidationHandler chain = new NoFlyListValidator();
chain.setNext(new CapacityValidator());
boolean valid = chain.validate(customerName, passengerNames, flights);

// Adding a new rule later: zero changes to existing code
chain.setNext(new CapacityValidator())
    .setNext(new AgeRestrictionValidator()) // new rule – just a new class
    .setNext(new RouteBlacklistValidator()); // another new rule
```

7. Code Quality Metrics Analysis

7.1 Metrics Used

The following standard Object-Oriented metrics are used, computed by manual analysis of the source code:

Metric	Description	Good Values
WMC (Weighted Methods per Class)	Total number of methods in a class. Higher = more complex, harder to maintain.	Lower is better
CBO (Coupling Between Objects)	Number of distinct other classes a class directly depends on. High CBO = fragile code.	Lower is better
RFC (Response For a Class)	Number of methods that can be executed in response to a message (direct + called). High RFC = complex, hard to test.	Lower is better

LCOM (Lack of Cohesion of Methods)	How unrelated the methods of a class are to each other. Higher = poor cohesion.	Lower is better
LOC (Lines of Code)	Source lines of code in the class body.	Contextual

Tool / Approach: Metrics computed by manual static analysis of the source files in `Reservation-System-Starter-master/src/main/java/`. The same criteria are applied before and after for a fair comparison.

7.2 Step 1 — Metrics Before Applying Patterns

Class	WMC	CBO	RFC	LCOM	LOC
FlightOrder	9	4	14	4	107
ScheduledFlight	9	5	12	5	85
Flight	5	5	8	3	68
Customer	7	3	10	4	79
PassengerPlane	1	0	1	0	33
Helicopter	3	0	3	0	25
PassengerDrone	1	0	1	0	13

Observations (Before):

- `FlightOrder` has WMC=9 and RFC=14 because it contains 5 payment-specific methods on top of order management — bloated and poorly cohesive.
- `ScheduledFlight` has CBO=5 because it directly imports and `instanceof`-checks `PassengerPlane`, `Helicopter`, and `PassengerDrone`. It is tightly coupled to all three concrete aircraft types.
- `Flight` has CBO=5 for the same reason — it depends on all 3 aircraft types via `instanceof`.
- `Customer` has a duplicated `isOrderValid()` that also appears in `FlightOrder` — evidenced by LCOM=4 (methods share little state).
- `PassengerPlane` fields are `public` (no encapsulation), directly accessed by `Flight` and `ScheduledFlight`.

7.3 Step 2 — Metrics After Applying Patterns

Class	WMC	CBO	RFC	LCOM	LOC	Pattern Applied
FlightOrder	4	2	6	1	40	Strategy
PaymentStrategy (<i>new</i>)	2	0	2	0	5	Strategy
CreditCardPayment (<i>new</i>)	2	1	4	0	30	Strategy
PayPalPayment (<i>new</i>)	2	1	3	0	25	Strategy
ScheduledFlight	7	2	9	1	65	Adapter + Observer

Flight	4	2	6	1	45	Adapter + Factory
Aircraft <i>(new)</i>	3	0	3	0	6	Adapter + Factory
AircraftFactory <i>(new)</i>	1	3	4	0	20	Factory
Customer	5	3	7	1	55	Observer + Builder + CoR
FlightOrderBuilder <i>(new)</i>	6	4	8	1	55	Builder
NoFlyListValidator <i>(new)</i>	1	1	2	0	25	Chain of Responsibility
CapacityValidator <i>(new)</i>	1	1	2	0	20	Chain of Responsibility
FlightObserver <i>(new)</i>	1	0	1	0	4	Observer

7.4 Step 3 — Side-by-Side Comparison (Key Classes)

Class	WMC Before → After	CBO Before → After	RFC Before → After	LCOM Before → After	LOC Before → After
FlightOrder	9 → 4 (↓56%)	4 → 2 (↓50%)	14 → 6 (↓57%)	4 → 1 (↓75%)	107 → 40 (↓63%)
ScheduledFlight	9 → 7 (↓22%)	5 → 2 (↓60%)	12 → 9 (↓25%)	5 → 1 (↓80%)	85 → 65 (↓24%)
Flight	5 → 4 (↓20%)	5 → 2 (↓60%)	8 → 6 (↓25%)	3 → 1 (↓67%)	68 → 45 (↓34%)
Customer	7 → 5 (↓29%)	3 → 3 (→)	10 → 7 (↓30%)	4 → 1 (↓75%)	79 → 55 (↓30%)

7.5 Step 3 — Analysis

FlightOrder — Most Improved Class

`FlightOrder` saw the most dramatic improvements driven by the **Strategy Pattern**. WMC dropped from 9 to 4 because 5 payment-specific methods (`processOrderWithCreditCard` , `processOrderWithCreditCardDetail` , `processOrderWithPayPal` , `payWithCreditCard` , `payWithPayPal`) were extracted into dedicated strategy classes. CBO fell from 4 to 2 as `FlightOrder` no longer directly imports `CreditCard` or `Paypal` — it only depends on the `PaymentStrategy` interface. RFC dropped from 14 to 6 for the same reason. LCOM improved from 4 to 1, confirming much higher cohesion — the remaining methods are all genuinely related to order management.

The LOC reduction from 107 to 40 lines is not a loss of functionality — the logic moved to `CreditCardPayment` and `PayPalPayment` , each of which is focused and independently testable.

ScheduledFlight and Flight — CBO Improvement

Both `ScheduledFlight` and `Flight` had CBO=5, directly depending on `PassengerPlane` , `Helicopter` , `PassengerDrone` , `Airport` , and one other class. After the **Adapter Pattern** introduced the `Aircraft` interface, both classes now depend only on `Aircraft` (interface) and `Airport` —

CBO=2 for each. This is a significant improvement: adding a new aircraft type no longer forces changes to these classes. LCOM improved dramatically (5→1 and 3→1) because the `instanceof` chains that fragmented method cohesion were eliminated.

WMC for `ScheduledFlight` slightly increased (9→7 after removing 3 `instanceof` methods = 6, but +1 for `registerObserver`, +1 for `removeObserver`, +1 for `notifyObservers` = 9 new WMC). This is a reasonable trade-off — the new methods represent genuine new functionality (notification), not complexity of existing logic.

Customer — DRY Improvement

`Customer` had a duplicated `isOrderValid()` method that was nearly identical to the one in `FlightOrder` — this was the most obvious DRY violation. After the **Chain of Responsibility Pattern** extracted validation into `NoFlyListValidator` and `CapacityValidator`, `Customer.isOrderValid()` became a 3-line chain call. LCOM improved from 4 to 1. CBO remains at 3 because `Customer` still depends on `FlightOrder`, `Order`, and `FlightObserver` (new — required by Observer Pattern).

The **Builder Pattern** further improved `Customer` by moving 7 of its 8 `createOrder()` steps into `FlightOrderBuilder.build()`, making `createOrder()` a clean 3-line delegation.

New Classes — Focused and Cohesive

All new classes introduced by the patterns (`PaymentStrategy`, `CreditCardPayment`, `PayPalPayment`, `Aircraft`, `AircraftFactory`, `FlightOrderBuilder`, `NoFlyListValidator`, `CapacityValidator`, `FlightObserver`) have low WMC (1–6), low CBO (0–4), and LCOM=0 or 1. This confirms that the patterns introduced well-focused classes, not additional complexity.

Metrics That Did Not Improve

`Customer.CBO` remained at 3. This is expected — `Customer` still needs to reference `FlightOrder`, `Order`, and `FlightObserver`. Adding the Observer Pattern introduced a new dependency, but it was offset by removing the duplicated validation dependencies. The net change is neutral.

`ScheduledFlight.WMC` increased by adding 3 observer methods — however, this represents added **functionality** (notification capability), not added complexity of existing logic.

Overall Conclusion

The six design patterns collectively improved the codebase across every major quality dimension:

- **Coupling (CBO)** reduced by 40–60% in the most problematic classes.
- **Complexity (WMC, RFC)** reduced by 20–57% in core classes.
- **Cohesion (LCOM)** improved by 67–80% in all refactored classes.
- **Code size (LOC)** reduced in core classes while total functionality increased.
- The **DRY principle** was restored by eliminating the duplicated `isOrderValid()` via Chain of Responsibility.
- The **Open/Closed Principle** is now observed in payment processing (Strategy), aircraft handling (Adapter/Factory), validation (Chain of Responsibility), and notification (Observer).

Summary

#	Pattern	Code Smell Addressed	Key Classes Changed	Key Benefit
1	Strategy	5 hardcoded payment methods in FlightOrder	New: PaymentStrategy, CreditCardPayment, PayPalPayment. Refactored: FlightOrder	OCP — add payment methods without modifying FlightOrder
2	Factory	Scattered new Aircraft() calls; Object type	New: AircraftFactory. Uses Aircraft interface from Section 3	Centralized, type-safe aircraft creation
3	Adapter	3 incompatible aircraft classes; instanceof chains in 3 methods	New: Aircraft interface. Refactored: PassengerPlane, Helicopter, PassengerDrone, Flight, ScheduledFlight	Unified interface; all instanceof chains eliminated
4	Observer	No notification when flight price/capacity changes	New: FlightObserver. Refactored: ScheduledFlight (subject), Customer (observer)	Real-time notifications; loose coupling between flights and customers
5	Builder	God Method createOrder() with 8 responsibilities	New: FlightOrderBuilder. Refactored: Customer.createOrder()	Readable, step-by-step construction; prevents invalid object state
6	Chain of Responsibility	isOrderValid() duplicated in Customer and FlightOrder; mixed validation concerns	New: OrderValidationHandler, NoFlyListValidator, CapacityValidator. Refactored: Customer	DRY — single chain; each rule is independent and extensible